ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ



БИЕ ДААЛТ 1
ТАЙЛАН

Үйлдлийн систем /F.CSM302/

Сэдэв: "The Producer-Consumer Problem. Programming Project 4 (7-р бүлэг), Buffer animation"

Хуваарилалтын код: II-4-12

Хичээл заасан багш: Г.Ганбат /F.CS10/

Гүйцэтгэсэн баг: А.Эрхэмтөр /B241940029/
Б.Хүслэн /B242270107/
Н.Батмөрөөдөл /B231910039/
Г.Оюунгэрэл /B232270802/
С.Итгэл /B242270031/

Огноо: 2026 оны 10 дугаар сарын 19

ХУРААНГУЙ

Асуудал. Producer-Consumer буюу хязгаартай буферийн (bounded buffer) бодлогод хэд хэдэн thread нэг дундын буферт зэрэг хандана. Producer дүүрсэн буферт, consumer хоосон буферээс хандах ёсгүй бөгөөд буферийг өөрчлөх хэсэгт нэг зэрэг хоёр thread орвол өгөгдөл алдагдана (race condition).

Шийдэл. Сурах бичгийн Programming Project 4-ийн дагуу C хэл, POSIX thread (pthreads), `empty`, `full` counting semaphore болон mutex lock ашиглан тойрог буферийг хэрэгжүүлэв. Программ үйлдэл бүрээ JSON мөрөөр хэвлэнэ. Next.js вэб интерфейс уг программыг ажиллуулж, бодит үйлдлүүдийг шууд хүлээн авч, багшийн оноосон "Buffer animation" дүрслэлээр алхам алхмаар харуулна. Интерфейс нь тохиргоо, бэлэн туршилт, timeline, event log, дуут хичээл, асуулттай түвшингүүдийг агуулна.

Туршилт. C программыг 18 автомат тестээр (үндсэн, захын, буруу оролт, дохио), timeline загварыг 3 тестээр шалгаж, GitHub Actions дээр Ubuntu ба macOS орчинд push бүрт ажиллуулдаг болгосон. Интерфейсийг бэлэн гурван туршилтаар гараар туршив.

Гол үр дүн. 18/18 тест амжилттай. Mutex-тэй бүх ажиллалтад "тавьсан - авсан = буферт үлдсэн" тэнцэл (invariant) биелсэн. Mutex-гүй ажиллалтын 5-аас 5-д нь race condition илэрсэн бөгөөд жишээ нь 1 секундын ажиллалтад 8708 үйлдлээс 584 зөрчил гарч, интерфейс бүрийг нь улаанаар тэмдэглэж, нэг нэгээр нь үзүүлсэн.

ГАРЧИГ

Хураангуй 1

НЭГ. *Удиртгал* 4
1.1 Зорилго 4
1.2 Ач холбогдол 4
1.3 Хамрах хүрээ 4

ХОЁР. *Сурах бичгийн шаардлага* 4
2.1 Programming Project 4-ийн үндсэн шаардлага 4
2.2 Багшийн нэмэлт шаардлага 5
2.3 Шаардлагын хангалт 5

ГУРАВ. *Онолын үндэс* 5
3.1 Critical section ба race condition 5
3.2 Semaphore 6
3.3 Mutex lock 6
3.4 Bounded buffer шийдэл 6
3.5 Тойрог буфер 6
3.6 Invariant 6

ДӨРӨВ. *Системийн зохиомж* 7
4.1 Бүрэлдэхүүн хэсгүүд 7
4.2 Өгөгдлийн урсгал 7
4.3 Үйлдлийн (event) формат 7
4.4 Хязгаар ба хамгаалалт 8

ТАВ. *Хэрэгжилт* 8
5.1 Хэл, сан, орчин 8
5.2 C core: буфер 9
5.3 C core: producer ба consumer 9
5.4 Сервер route 10
5.5 Интерфейсийн загвар 10
5.6 Интерфейс 10
5.7 GitHub дээрх ажиллагаа 11
5.8 Ашигласан хэрэгсэл ба бэлэн код 12

ЗУРГАА. *Туршилт ба UI дүрслэл* 12
6.1 Өөрчилж турших параметр 12
6.2 Юуг ажиглах 13
6.3 Бэлэн туршилтууд 13

ДОЛОО. *Туршилтын арга* 13
7.1 Орчин 13
7.2 Core-ийн автомат тест 13
7.3 Загварын тест ба CI 14
7.4 Гар туршилт 14

НАЙМ. *Үр дүн* 14
8.1 Core-ийн тестийн үр дүн 14
8.2 Интерфейсийн үр дүн 15

ЕС. *Хэлэлцүүлэг* 17
9.1 Үр дүнгийн тайлбар 17
9.2 Хүлээх дараалал ба deadlock 18
9.3 Хязгаарлалт 18

АРАВ. *Дүгнэлт* 18

Ашигласан материал 18

Хавсралт А. Ажлын төлөвлөгөө 19

НЭГ. УДИРТГАЛ

1.1 Зорилго

Сурах бичгийн 7-р бүлгийн Programming Project 4 "The Producer-Consumer Problem"-ийг хэрэгжүүлж, хэрэглэгч өөр өгөгдөл, параметрээр турших боломжтой программ бүтээх. Программын ажиллагааг багшийн оноосон "Buffer animation" дүрслэлээр харуулж, хэрэгжилт болон туршилтын үр дүнг баримтжуулах.

1.2 Ач холбогдол

Producer-Consumer нь синхрончлолын сонгодог бодлого бөгөөд үйлдлийн системд гарын товчлуурын буфер, принтерийн дараалал, pipe (`ls | grep`), сүлжээний пакетын буфер, видео урьдчилан ачаалах зэрэг олон газарт ашиглагддаг. Semaphore ба mutex-ийг зөв дарааллаар хэрэглэх, race condition-ийг таних нь зэрэгцээ программчлалын үндсэн чадвар юм. Thread-үүдийн ажиллагаа нүдэнд харагддаггүй тул бодит программын үйлдлийг дүрслэх нь ойлголтыг ихээхэн хөнгөвчилнө.

1.3 Хамрах хүрээ

- C хэл дээрх producer/consumer thread, тойрог буфер, `empty`/`full` semaphore, mutex (сурах бичгийн үндсэн шаардлага).
- Mutex-гүй горимоор race condition-ийг санаатайгаар үүсгэж харуулах.
- Оролтыг шалгах, үйлдэл бүрийг JSON мөрөөр хэвлэх, дохиогоор (SIGTERM) зогсох.
- Бодит программтай шууд холбогдсон вэб интерфейс: тохиргоо, дүрслэл, timeline, event log, хичээл, түвшингүүд.
- Автомат тест, CI, хэрэглэгчийн ба operational гарын авлага (Manual).

Олон процесс (fork) хоорондын буфер, Windows API хувилбар хамрах хүрээнд ороогүй.

ХОЁР. СУРАХ БИЧГИЙН ШААРДЛАГА

2.1 Programming Project 4-ийн үндсэн шаардлага

Сурах бичгийн (Silberschatz, Galvin, Gagne, *Operating System Concepts*, 10-р хэвлэл) 7-р бүлгийн Project 4 нь 7.1.1 хэсгийн bounded buffer шийдлийг хэрэгжүүлэхийг шаардана:

1. Буфер нь `buffer_item` (`int`) төрлийн, тогтмол хэмжээтэй (номд 5) тойрог массив байх, `insert_item()`, `remove_item()` функцээр хандах.
2. Producer thread санамсаргүй хугацаагаар унтаад, санамсаргүй тоо үүсгэн буферт оруулна. Consumer thread санамсаргүй хугацаагаар унтаад, буферээс нэгийг авна.
3. `main()` нь командын мөрөөс 3 аргумент авна: хэдэн секунд ажиллах, producer-ийн тоо, consumer-ийн тоо. Thread-үүдээ үүсгээд тэр хугацаанд унтаж, программыг дуусгана.

4. `empty`, `full`-ийг counting semaphore-оор, буферийн хамгаалалтыг mutex lock-оор хэрэгжүүлэх. Pthreads эсвэл Windows API ашиглаж болно.

2.2 Багшийн нэмэлт шаардлага

Бие даалт 1-ийн удирдамжаар дараах шаардлага нэмэгдэнэ: оролт, параметрийг өөрчилж дахин турших; буруу болон захын оролтыг шалгаж ойлгомжтой мэдээлэх; тохиргоог солих, дахин ажиллуулах, reset хийх; оноосон "Buffer animation" UI-г бодит ажиллагаатай холбох; номын болон нэмэлт туршилтаар зөв ажиллагааг нотлох; Manual-ын дагуу өөр компьютерт ажиллуулах боломжтой байх; GitHub repository, README, Release/Tag.

2.3 Шаардлагын хангалт

Хүснэгт 1. Шаардлага ба хэрэгжсэн газар

Шаардлага	Хэрэгжилт
Тойрог буфер, `insert_item`/`remove_item`	`src/core/pc.c`, `in`/`out` заагч, `% bsize`
Producer/consumer thread, санамсаргүй унталт	`pthread_create`, `rand_r` (thread бүр өөрийн seed)
3 аргумент, нэмэлт сонголт	`pc <сек> <P> <C> [--buffer N] [--sleep-max MS] [--seed S] [--no-mutex]`
`empty`, `full` semaphore, mutex	POSIX semaphore (macOS дээр `sem_open`), `pthread_mutex_t`
Параметр өөрчилж дахин турших, reset	Вэб интерфейсийн Тохиргоо цонх, Start/Stop/Reset, бэлэн туршилт
Буруу оролт шалгах	C программ (exit код + тайлбар), `/api/run` (HTTP 400), UI (улаан талбар, алдааны мэдээлэл)
Buffer animation, бодит ажиллагаатай холбох	Бодит программын үйлдлийг шууд stream хийж дүрслэх: Массив, Хоолой, Талх дүрслэл
Нэмэлт туршилт	18 автомат тест, timeline загварын тест, CI, race туршилт

ГУРАВ. ОНОЛЫН ҮНДЭС

3.1 Critical section ба race condition

Хэд хэдэн thread нэг дундын өгөгдлийг зэрэг өөрчлөх үед үр дүн нь гүйцэтгэлийн дарааллаас хамаарах нөхцөлийг race condition гэнэ. Буферт тавих үйлдэл нь `"buffer[in] = item"`, `"in = (in + 1) % n"`, `"count++"` гэсэн хэд хэдэн алхмаас бүрдэх бөгөөд энэ хэсгийг critical section гэнэ. Хоёр producer зэрэг орвол нэг нүд рүү хоёулаа бичиж, нэг утга алга болох, `count` буруу болох аюултай. Critical section-ийн шийдэл нь харилцан гадуурхал (mutual exclusion), ахиц (progress), хязгаартай хүлээлт (bounded waiting) гэсэн

гурван шаардлагыг хангах ёстой.

3.2 Semaphore

Semaphore нь бүхэл тоон утгатай хувьсагч бөгөөд зөвхөн хоёр атомар үйлдлээр хандана. `wait()` (POSIX: `sem_wait`) утгыг 1-ээр хасна; утга 0 байвал thread блоклогдож хүлээнэ. `signal()` (`sem_post`) утгыг 1-ээр нэмж, хүлээж буй thread байвал нэгийг сэрээнэ. Counting semaphore нь нөөцийн тоог хадгалдаг тул буферийн сул болон дүүрсэн нүдний тоог тоолоход тохиромжтой.

3.3 Mutex lock

Mutex нь нэг мөчид зөвхөн нэг thread-д critical section-д орох эрх олгоно (`pthread_mutex_lock`/`unlock`). Цоож барьсан thread л буферийг өөрчлөх тул `in`, `out`, `count` зөрөхгүй.

3.4 Bounded buffer шийдэл

Номын 7.1.1 хэсгийн шийдэл гурван синхрончлолын хувьсагч ашиглана: `mutex = 1`, `empty = n` (сул нүдний тоо), `full = 0` (дүүрсэн нүдний тоо). Producer-ийн бүтэц (номын Зураг 7.1):

Код 1. Producer-ийн бүтэц (номын Зураг 7.1)

```
while (true) {
   /* produce an item in next_produced */
   wait(empty);
   wait(mutex);
   /* add next_produced to the buffer */
   signal(mutex);
   signal(full);
}
```

Consumer нь тэгш хэмтэй: `wait(full)`, `wait(mutex)`, авах, `signal(mutex)`, `signal(empty)`. `empty` нь буфер дүүрэх үед producer-ийг, `full` нь хоосон үед consumer-ийг зогсооно. Mutex нь зөвхөн буферийг өөрчлөх богино хэсгийг хамгаална.

3.5 Тойрог буфер

Буфер нь `n` нүдтэй массив, хоёр заагчтай: `in` нь дараагийн тавих нүд, `out` нь дараагийн авах нүд. Заагч бүр `(i + 1) % n`-ээр урагшилж, сүүлийн нүдний дараа 0 руу эргэнэ. Иймээс анх тавьсан нь анх авагдана (FIFO).

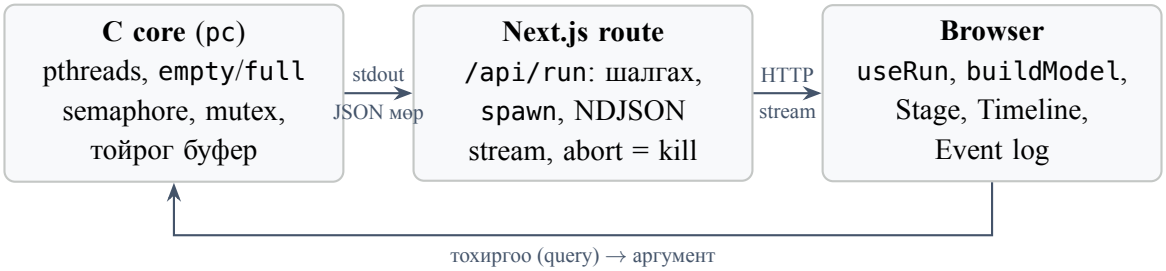
3.6 Invariant

Зөв синхрончлолтой ажиллалтын төгсгөлд "тавьсан тоо - авсан тоо = буферт үлдсэн тоо" ба `0 <= count <= n` үргэлж биелнэ. Энэ тэнцлийг программ төгсгөлд нь шалгаж (`invariant_ok`), тест болон интерфейс зөв ажиллагааны нотолгоо болгон ашиглана.

ДӨРӨВ. СИСТЕМИЙН ЗОХИОМЖ

4.1 Бүрэлдэхүүн хэсгүүд

Систем гурван хэсгээс бүрдэнэ (Зураг 1): синхрончлолыг бодитоор гүйцэтгэдэг C программ (core), түүнийг ажиллуулж үйлдлийг дамжуулдаг Next.js сервер route, үйлдлийг дүрсэлдэг browser талын интерфейс.



Зураг 1. Системийн бүрэлдэхүүн ба өгөгдлийн урсгал

4.2 Өгөгдлийн урсгал

1. Хэрэглэгч Тохиргоо цонхонд producer, consumer, буфер, унтах хугацаа, ажиллах хугацаа, seed, mutex-ийг сонгож Start дарна.
2. Browser `GET /api/run?...` хүсэлт илгээнэ. Route нь утгыг шалгаж (буруу бол HTTP 400 + тайлбар), `pc`-г аргументтай нь `spawn` хийнэ.
3. Core thread-үүдээ эхлүүлж, үйлдэл бүрийг (унтах, хүлээх, тавих, авах) нэг JSON мөрөөр stdout руу хэвлэнэ. Route мөр бүрийг шууд browser руу дамжуулна (NDJSON stream).
4. Browser мөрүүдийг хүлээн авч загвар (`buildModel`) үүсгэн, Stage, Timeline, Event log-д тоглуулна. Хэрэглэгч Stop дарах эсвэл хуудаснаас гарвал хүсэлт тасарч, route core процессыг `SIGTERM`-ээр зогсооно.

4.3 Үйлдлийн (event) формат

Core-ийн мөр бүр нэг үйлдлийг илэрхийлэх бөгөөд тухайн агшны буферийн төлвийг (snapshot) агуулна (Хүснэгт 2).

Хүснэгт 2. JSON үйлдлийн талбарууд

Талбар	Утга
`t`	core эхэлснээс хойших хугацаа (мс)
`ev`	`start`, `sleep`, `wait_empty`, `wait_full`, `produce`, `consume`, `error`, `end`
`role`, `id`	`producer`/`consumer` ба 1-ээс эхлэх дугаар
`item`, `slot`	утга ба нүдний индекс
`empty`, `full`	semaphore-ын утга
`in`, `out`, `count`	буферийн заагчууд ба доторх элементийн тоо
`mtx`	mutex эзэмшигч (жишээ нь `"P2"`)
`end`	`produced`, `consumed`, `in_buffer`, `invariant_ok`

Жишээ мөр:

```
{"t":16,"ev":"produce","role":"producer","id":1,"item":57,"slot":0,
 "empty":2,"full":1,"in":1,"out":0,"count":1,"mtx":"P1"}
```

4.4 Хязгаар ба хамгаалалт

Route нь нэг зэрэг хамгийн ихдээ 4 core ажиллуулна (HTTP 429), өөр сайтаас ирсэн хүсэлтийг татгалзана (HTTP 403), нэг ажиллалтад 20000-аас олон үйлдэл гарвал core-г зогсоож анхааруулна. Интерфейсийн хязгаар: producer, consumer 0-16, буфер 1-16, унтах 0-5000 мс, хугацаа 1-60 с.

ТАВ. ХЭРЭГЖИЛТ

5.1 Хэл, сан, орчин

Хүснэгт 3. Ашигласан технологи

Хэсэг	Технологи
Core	C (gcc/clang, `-O2 -Wall -Wextra -pthread`), POSIX threads, POSIX semaphore
Сервер	Node.js 20+, Next.js 16.3.8 App Router route (`child_process.spawn`)
Интерфейс	React 19.3, TypeScript 5.9, Tailwind CSS 4.3, shadcn/ui, lucide-react
Тест	Python 3 (`tests/run_tests.py`), `node --test`
CI/CD	GitHub Actions (Ubuntu, macOS), main руу push бүрт автомат Release

5.2 C core: буфер

Номын API-г хадгалж, тойрог буферийг дараах байдлаар хэрэгжүүлэв. Функцүүд mutex дотроос дуудагдана.

Код 2. `insert_item` ба `remove_item` (src/core/pc.c)

```
static int insert_item(buffer_item item, int *slot) {
    if (count >= bsize) return -1;
    *slot = in_idx;
    buffer[in_idx] = item;
    in_idx = (in_idx + 1) % bsize;
    count++;
    return 0;
}

static int remove_item(buffer_item *item, int *slot) {
    if (count <= 0) return -1;
    *slot = out_idx;
    *item = buffer[out_idx];
    out_idx = (out_idx + 1) % bsize;
    count--;
    return 0;
}
```

5.3 C core: producer ба consumer

Producer санамсаргүй унтаад (`rand_r`, thread бүр seed-ээс гарган авсан өөрийн төлөвтэй, ижил seed = ижил унталт), 0-99 тоо үүсгэнэ. Хүлээлтийг интерфейст харуулахын тулд эхлээд `sem_trywait`-ээр оролдож, амжилтгүй бол `wait_empty` үйлдэл хэвлээд жинхэнэ `sem_wait`-ээр блоклогдоно. Дараа нь mutex барьж буферт тавина.

Код 3. Producer-ийн гол хэсэг (src/core/pc.c, товчилсон)

```
if (sem_trywait(sem_empty) != 0) {      /* buffer full -> will block */
    emit("wait_empty", "producer", id, item, -1, -1);
    sem_wait_retry(sem_empty);
}
if (use_mutex) { pthread_mutex_lock(&mutex); mtx_owner = id; }
if (insert_item(item, &slot) == 0)
    emit("produce", "producer", id, item, slot, -1);
if (use_mutex) { mtx_owner = 0; pthread_mutex_unlock(&mutex); }
sem_post(sem_full);
```

Consumer нь `full` дээр хүлээж, `remove_item`-ээр авч, `sem_post(empty)` хийнэ. `--no-mutex` сонголтоор mutex-ийг алгасаж race condition-ийг санаатайгаар үүсгэнэ. Бусад шийдэл:

- macOS нэргүй semaphore (`sem_init`) дэмждэггүй тул `#ifdef __APPLE__` үед

`sem_open` ашиглана. `sem_getvalue` macOS дээр ажиллахгүй тул `empty`/`full`-ийн утгыг semaphore-той зэрэг атомар тоолууураар хадгална.
- Хэвлэлтийг тусдаа `log_mutex`-ээр хамгаалж мөрүүд холилдохгүй. `produce`/`consume` нь mutex дотроос хэвлэгддэг тул mutex-тэй үед лог дахь дараалал нь буферт болсон бодит дараалал.
- `SIGINT`/`SIGTERM`-д `running = 0` болгож, блоклогдсон thread-үүдийг `sem_post`-оор сэрээж, `pthread_join`-оор хүлээгээд, `end` мөрөнд invariant-ийг хэвлэнэ.
- Аргумент бүрийг хүрээгээр шалгаж (жишээ нь буфер 1-64), алдаатай бол Монгол хэлээр тайлбар, хэрэглээний заавар хэвлээд 1 кодоор гарна.

5.4 Сервер route

`src/app/api/run/route.ts` нь query-г `RunConfig` болгон шалгаж, `pc`-г `spawn` хийн stdout-ийн мөр бүрийг `ReadableStream`-ээр дамжуулна. Хүсэлт тасрахад (Stop, хуудаснаас гарах) core-г зогсоож, stream-ийг хаана. Процесс дуусахад `{"ev":"closed","code":...}` мөр нэмж илгээнэ.

5.5 Интерфейсийн загвар

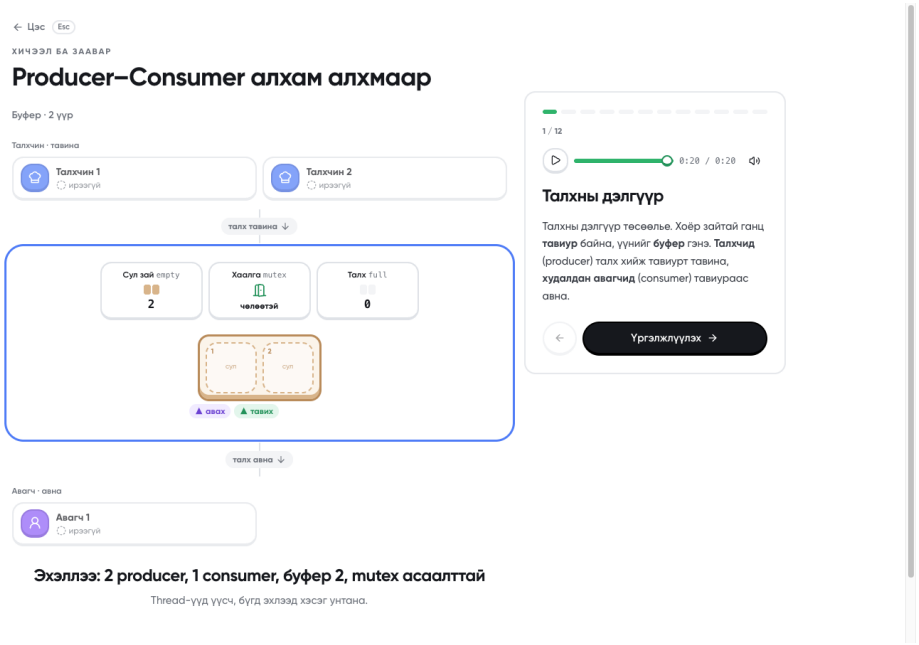
`useRun` hook stream-ийг уншиж үйлдлүүдийг хуримтлуулна. `buildModel` (`src/lib/model.ts`) нь thread бүрийн timeline-ий хэсгүүд (унтах, хүлээх, тавих, авах), mutex эзэмшил, буферийн төлөвийг гаргана. Race-ийг дараах байдлаар илрүүлнэ: `produce`/`consume` мөрийн `count` нь тэр хүртэлх "тавьсан - авсан"-тай тэнцэх ёстой. Зөрвөл тэр агшинд өөр thread зэрэг critical section-д байсан гэсэн үг тул race гэж тэмдэглээд дахин тааруулна. Mutex-тэй үед энэ нь үргэлж тэнцүү байдаг нь тестээр батлагдсан.

5.6 Интерфейс

- **Эхлэлийн цэс**: Хичээл ба заавар, Түвшингүүд, Тухай (Зураг 2).
- **Stage**: producer, consumer тус бүр карт (өнгө, төлөв, гартаа барьсан утга, mutex цоож), буфер гурван дүрслэлээр (Массив: нүд ба `in`/`out`; Хоолой: FIFO дараалал; Талх: талхны тавиур), тэдгээрийн хооронд "тавина/авна" сум, `empty`/`full`/mutex утга, тайлбар мөр.
- **Timeline** (thread бүр нэг мөр), **Event log** (JSONL татах), тоглуулагч (slider, хурд 0.1-1x, алхам).
- **Хичээл ба заавар**: 3 оршил, 12 алхам; талхны дэлгүүрийн жишээнээс эхлээд бодит run-ийг алхам алхмаар тоглуулж, дуу хоолойгоор уншина, 5 асуулттай (Зураг 3).
- **Түвшингүүд**: Буфер дүүрэх, Буфер хоосрох, Race condition; run дуусахад асуулт гарна.



Зураг 2. Эхлэлийн цэс. Дээрх 5 нүд нь бодит core-ийн жижиг run



Зураг 3. Хичээл ба заавар: талхны дэлгүүрийн жишээгээр эхэлж, алхам бүрийг уншина

5.7 GitHub дээрх ажиллагаа

Repository: `https://github.com/khuslenshi/os-bd1-producer-consumer`. Ажлыг branch-аар хийж (`ci-release`, `ui-nextjs`, `tests-extend`), CI болон release-ийн өөрчлөлтийг Pull Request #1-ээр нэгтгэв. Дараа нь хөгжүүлэлтийг `dev` branch-д төвлөрүүлж, анхны `main`-ийг `recovery` branch-д хадгалаад `dev`-ийг `main` руу нэгтгэсэн. GitHub Actions push ба Pull Request бүрт Ubuntu, macOS дээр C build (`-Werror`), 18 тест, загварын тест, Next.js build, `/api/run` smoke тестийг ажиллуулна. `main` руу push бүрт CI амжилттай бол шинэ хувилбар автоматаар тооцогдож (`fix` бол patch, `feat` бол minor, эхнийх нь `v1.0.0`), tag болон GitHub Release тестийн үр дүнтэйгээ гарна.

5.8 Ашигласан хэрэгсэл ба бэлэн код

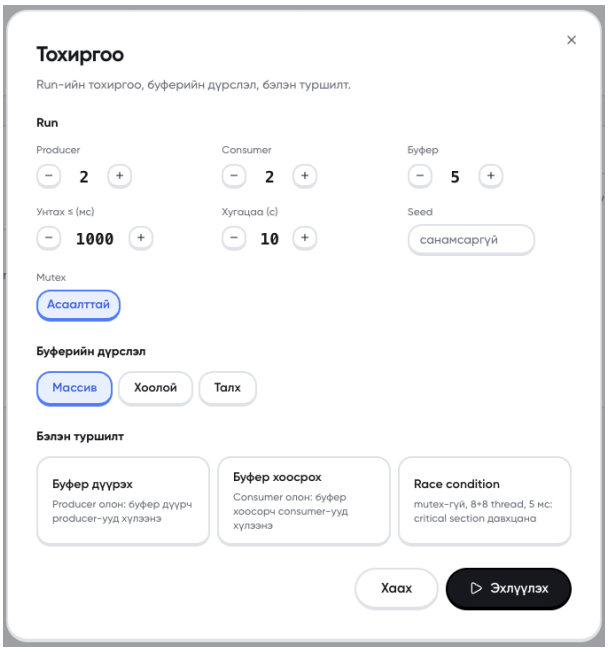
Next.js интерфейс, CI/release workflow, Manual-ын ихэнх хэсгийг Claude Code (Anthropic)-ийн тусламжтайгаар бичиж, өөрчлөлт бүрийг бодит C core дээр ажиллуулан тест, CI болон гараар шалгасан. Хичээлийн уншлагыг ElevenLabs TTS-ээр дуу болгосон. Бэлэн код: shadcn/ui component-ууд (өөрсдийн загвараар өөрчилсөн), lucide-react icon. Дэлгэрэнгүйг README-ийн "Тэмдэглэл" хэсгээс харна.

ЗУРГАА. ТУРШИЛТ БА UI ДҮРСЛЭЛ

6.1 Өөрчилж турших параметр

Хүснэгт 4. Тохиргооны параметр

Параметр	Нөлөө	Хүрээ (анхдагч)
Producer, Consumer	thread-ийн тоо; харьцаа нь буфер дүүрэх эсвэл хоосрохыг тодорхойлно	0-16 (2, 2)
Буфер	нүдний тоо; бага бол хүлээлт олширно	1-16 (5)
Унтах (мс)	үйлдэл хоорондын санамсаргүй хугацааны дээд хязгаар; бага бол contention ихэснэ	0-5000 (1000)
Хугацаа (с)	ажиллах хугацаа	1-60 (10)
Seed	ижил seed = ижил унталтын дараалал	хоосон = санамсаргүй
Mutex	унтраавал race condition гарна	асаалттай



Зураг 4. Тохиргоо цонх: параметр, буферийн дүрслэл, бэлэн туршилт

6.2 Юуг ажиглах

- Буфер дүүрэхэд (`empty = 0`) producer-ийн карт шар болж "хүлээж байна", consumer нэгийг авмагц хүлээж байсан producer сэрж тавина.
- Буфер хоосон үед (`full = 0`) consumer-ууд шар болж хүлээнэ.
- Mutex эзэмшигч хар хүрээ, цоожтой харагдана; нэг мөчид зөвхөн нэг.
- `in`, `out` заагч урагшилж, сүүлийн нүдний дараа 0 руу эргэнэ (FIFO).
- Mutex-гүй үед race гарсан үйлдэл улаан болж, "count = 2, 1 байх ёстой" гэх мэт тайлбар гарна. Ажиллалтын төгсгөлд invariant шалгагдана.

6.3 Бэлэн туршилтууд

Хүснэгт 5. Интерфейсийн бэлэн туршилт

Туршилт	Тохиргоо	Хүлээгдсэн ажиглалт
Буфер дүүрэх	P=4, C=1, буфер=2, 600 мс, 8 с, seed 7	producer-ууд `empty = 0` дээр хүлээнэ
Буфер хоосрох	P=1, C=4, буфер=5, 600 мс, 8 с, seed 7	consumer-ууд `full = 0` дээр хүлээнэ
Race condition	P=8, C=8, буфер=5, 5 мс, 1 с, mutex-гүй	олон race, invariant зөрөх боломжтой

ДОЛОО. ТУРШИЛТЫН АРГА

7.1 Орчин

Хөгжүүлэлт ба гар туршилт: macOS (Apple Silicon), Apple clang 21, Node.js 26. Автомат туршилт: GitHub Actions `ubuntu-latest` (gcc) ба `macos-latest` (clang), Node.js 24. Windows дээр WSL2 Ubuntu-аар ажиллана.

7.2 Core-ийн автомат тест

`tests/run_tests.py` нь `pc`-г олон тохиргоогоор ажиллуулж, JSON гаралтыг задлан шалгана:

- **Үндсэн (T1-T4)**: invariant, FIFO, нүдний дараалал; буфер дүүрэх ба хоосрох үед хүлээлт гарах.
- **Захын (B1-B6)**: буфер=1, унтах=0 (хамгийн их contention), 100+100 thread, буфер=64, 0 consumer, 0 producer.
- **Буруу оролт (E1-E7)**: аргумент дутуу, сөрөг хугацаа, тоо биш утга, буфер=0, хэт олон thread, үл мэдэгдэх сонголт, утгагүй сонголт. Exit код ба алдааны текстийг шалгана.
- **Дохио (S1)**: 60 секундын ажиллалтыг `SIGTERM`-ээр 1 секундэд зогсоож, `end` мөр гарах эсэх.
- **Race**: mutex-гүй горимыг 5 удаа ажиллуулж зөрчил илрэх эсэх (мэдээллийн

зорилготой).

7.3 Загварын тест ба CI

`src/lib/model.test.ts`: зохиомол лог дээр timeline-ий хэсэг, буферийн төлөв зөв гарах; mutex-гүй логт race нэг удаа тэмдэглэгдэж дахин тааруулах; бодит core-оор mutex-тэй үед 0 race, mutex-гүй үед race байгаа ч үйлдлийн тооноос хамаагүй бага байх. CI нь эдгээрээс гадна production build хийж, `/api/run`-аар бодит run-ийг stream хийн `"invariant_ok":true` гарах, `buffer=0` хүсэлтэд 400 буцах, run-ийг дундаас нь таслахад сервер алдаа гаргахгүй байхыг шалгана.

7.4 Гар туршилт

Интерфейсийн бэлэн гурван туршилт, буруу оролт (буфер 0), хичээлийн 12 алхам, гурван түвшний асуултыг browser дээр гараар туршиж, дэлгэцийн зургаар баримтжуулав (8-р бүлэг).

НАЙМ. ҮР ДҮН

8.1 Core-ийн тестийн үр дүн

macOS дээрх ажиллалтын үр дүн (Хүснэгт 6). Ubuntu дээр CI-д мөн бүгд амжилттай.

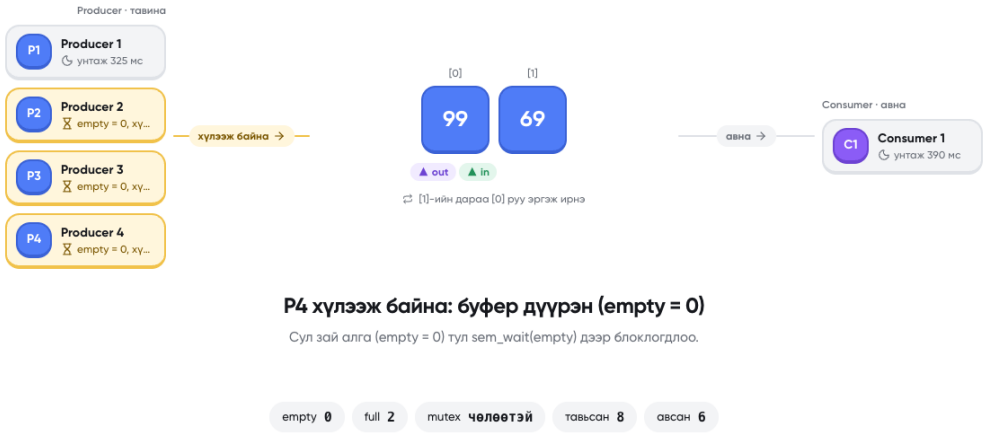
Хүснэгт 6. Автомат тестийн үр дүн (`tests/results.md`)

ID	Тест	Бодит үр дүн	Дүн
T1	1P+1C, 3 с, буфер=5	тавьсан 26, авсан 22, үлдсэн 4	PASS
T2	3P+1C: буфер дүүрэх	57/52/5, хүлээлт (empty) 48	PASS
T3	1P+4C: буфер хоосрох	52/52/0, хүлээлт (full) 55	PASS
T4	Invariant, 5 с	501 - 499 = 2	PASS
B1	буфер=1	164/164/0, нүд үргэлж 0	PASS
B2	унтах=0	66337/66337/0	PASS
B3	100P+100C	24461/24460/1	PASS
B4	буфер=64	315/251/64	PASS
B5	0 consumer	5/0/5, producer гацна	PASS
B6	0 producer	0/0/0, consumer хүлээнэ	PASS
E1-E7	буруу оролт	exit=1, тохирох тайлбар (жишээ нь "buffer нь 1..64 хооронд байх ёстой")	PASS
S1	SIGTERM	1.0 с-д зогсож `end` гарсан	PASS

Нийт 18/18 PASS. Загварын 3 тест PASS. Mutex-гүй 5 ажиллалтын 5-д race илэрсэн.

8.2 Интерфейсийн үр дүн

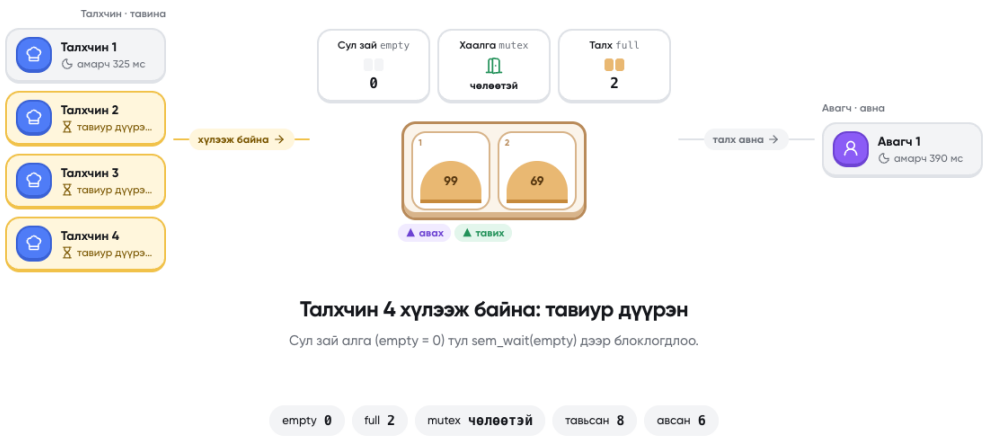
Зураг 5-11 нь бодит программын ажиллалтаас авсан дэлгэцийн зураг.



Зураг 5. Буфер дүүрэх: P2-P4 `empty = 0` дээр хүлээж, "хүлээж байна" сум шар



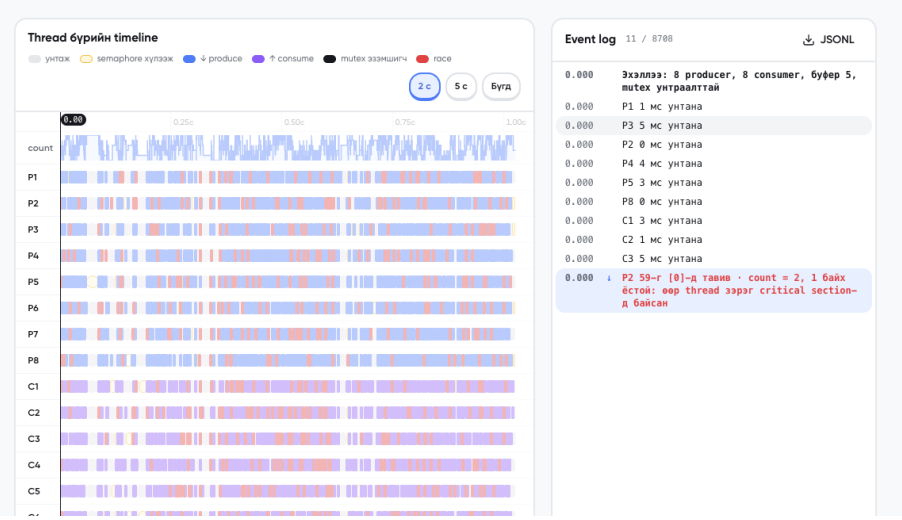
Зураг 6. Буфер хоосрох (Хоолой дүрслэл): C2-C4 `full = 0` дээр хүлээж байна



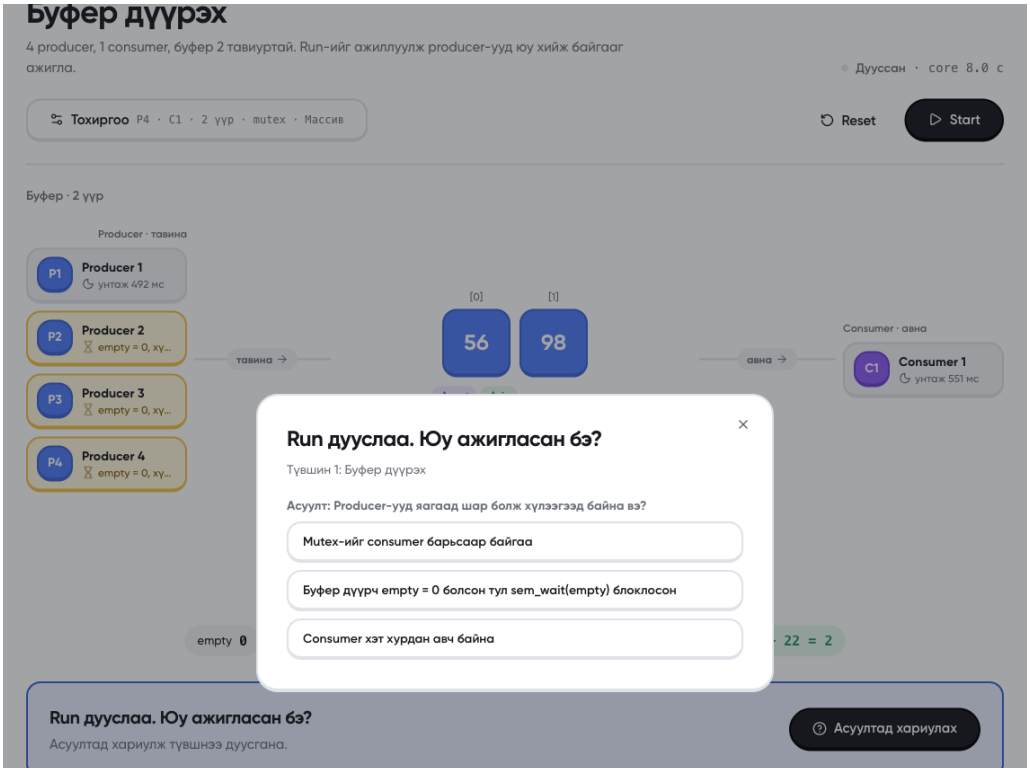
Зураг 7. Талх дүрслэл: тавиур дүүрэн, Сул зай 0, Талх 2



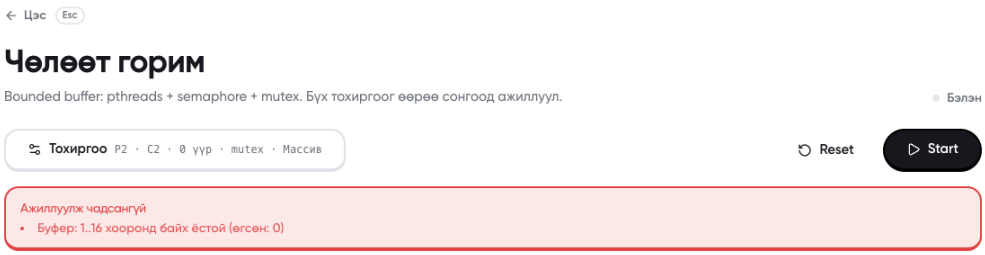
Зураг 8. Race condition: 1 секундэд 8708 үйлдэл, эхний race дээр зогссон үе. "count = 2, 1 байх ёстой"



Зураг 9. Race-ийн timeline (улаан тэмдэг) ба Event log



Зураг 10. Түвшин 1 дууссаны дараа: Invariant OK (24 - 22 = 2), асуулт



Зураг 11. Буруу оролт: буфер 0 үед алдааны мэдээлэл

ЕС. ХЭЛЭЛЦҮҮЛЭГ

9.1 Үр дүнгийн тайлбар

Mutex-тэй бүх тест, тэр дундаа унтах хугацаа 0 (66337 үйлдэл) ба 100+100 thread гэх мэт хамгийн их contention-тэй нөхцөлд invariant биелсэн нь semaphore ба mutex-ийн хослол буферийг зөв хамгаалж байгааг харуулна. Producer олон үед `empty` (T2: 48 хүлээлт), consumer олон үед `full` (T3: 55 хүлээлт) semaphore thread-үүдийг зогсоож байгаа нь интерфейс дээр шар картаар шууд харагдана. Mutex-гүй үед race тогтмол гарч, зарим ажиллалтад төгсгөлийн invariant ч зөрөх боломжтой нь "lost update"-ийн бодит жишээ юм.

9.2 Хүлээх дараалал ба deadlock

Producer эхлээд `wait(empty)`, дараа нь `wait(mutex)` хийх ёстой. Хэрэв дарааллыг солиод mutex-ийг барьсан чигтээ дүүрэн буфер дээр `wait(empty)` хийвэл consumer mutex авч чадахгүй тул хэн ч урагшлахгүй (deadlock). Манай хэрэгжилт номын дарааллыг баримталсан тул ийм гацаа гарахгүй. Мөн mutex-ийг зөвхөн буферийг өөрчлөх богино хэсэгт барьж, унталт ба хүлээлтийг critical section-ээс гадуур хийсэн.

9.3 Хязгаарлалт

- Хүлээлтийг `sem_trywait`-ээр урьдчилан илрүүлдэг тул `wait_*` үйлдэл нь "блоклогдох гэж байна" гэсэн утгатай; маш богино хугацаанд сэрвэл хүлээлт бараг харагдахгүй.
- Mutex-гүй үед лог дахь дараалал найдваргүй тул race-ийг зөвхөн `count`-ын зөрүүгээр илрүүлдэг; нэг агшинд хэд хэдэн thread давхцсан ч нэг race гэж тоологдож болно.
- Нэг ажиллалт хамгийн ихдээ 20000 үйлдэл; маш олон үйлдэлтэй ажиллалтыг (Түвшин 3) нүдээр хөөх боломжгүй тул интерфейс анхааруулга гаргаж, race-уудыг нэг нэгээр нь үзүүлнэ.
- Интерфейсийн хязгаар (thread 16, буфер 16) core-оос (thread 200, буфер 64) нарийн.

АРАВ. ДҮГНЭЛТ

Сурах бичгийн Programming Project 4-ийг C хэл, pthreads, POSIX semaphore, mutex ашиглан бүрэн хэрэгжүүлж, буферийн хэмжээ, thread-ийн тоо, унтах хугацаа, seed, mutex-ийг өөрчилж турших боломжтой болгосон. Багшийн оноосон "Buffer animation" дүрслэлийг бодит программын үйлдэлтэй шууд холбож, гурван дүрслэл, timeline, event log, дуут хичээл, асуулттай түвшингүүдээр харуулав. 18 автомат тест, загварын тест, Ubuntu ба macOS дээрх CI-аар зөв ажиллагааг нотолж, mutex-гүй үед race condition бодитоор гарч, илрүүлэгдэж байгааг харуулсан. Manual-ын дагуу `npm install`, `npm run dev` гэсэн хоёр командаар өөр компьютерт ажиллуулах боломжтой.

АШИГЛАСАН МАТЕРИАЛ

[1] A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed. Wiley, 2018. Chapter 6-7, Programming Project 4 "The Producer-Consumer Problem".

[2] The Open Group. POSIX.1-2017: `pthread_create`, `pthread_mutex_lock`, `sem_wait`, `sem_post`, `sem_open`.

[3] Next.js Documentation: Route Handlers, streaming. `https://nextjs.org/docs`

[4] shadcn/ui. `https://ui.shadcn.com`

[5] F.CSM302 Бие даалт 1 удирдамж, Сэдэв ба шүүмжийн хуваарилалт.

[6] Багийн repository. `github.com/khuslenshi/os-bd1-producer-consumer`

ХАВСРАЛТ А. АЖЛЫН ТӨЛӨВЛӨГӨӨ

Талбар	Мэдээлэл
Хуваарилалтын код	II-4-12
Төсөл	The Producer-Consumer Problem
Repository	`https://github.com/khuslenshi/os-bd1-producer-consumer`
Хэл/сан/орчин	C (pthreads, POSIX semaphore), Node.js, Next.js, TypeScript; macOS, Linux, WSL2
Үндсэн дүрслэл	Buffer animation

Гишүүн	Хариуцах ажил	Commit/file
А.Эрхэмтөр /B241940029/	Next.js интерфейс, CI/release, Manual, тайлан	`src/app`, `src/components`, `.github/`, `docs/`
Б.Хүслэн /B242270107/	C core, анхны интерфейс, тест	`src/core/pc.c`, `tests/`
Н.Батмөрөөдөл /B231910039/		
Г.Оюунгэрэл /B232270802/	Нэмэлт тест	`tests-extend` branch
С.Итгэл /B242270031/		

Тест	Оролт/нөхцөл	Хүлээгдсэн үр дүн
1	P=1, C=1, буфер=5, 3 с	invariant, FIFO, нүдний дараалал зөв
2	P=3, C=1 (буфер дүүрэх)	producer `empty` дээр хүлээнэ
3	P=1, C=4 (буфер хоосрох)	consumer `full` дээр хүлээнэ
4	буфер=1, унтах=0, 100+100 thread	захын нөхцөлд invariant биелнэ
5	`--no-mutex`, P=8, C=8, 5 мс	race илэрнэ